Indirect Communication

Distributed Software Systems

CdLM Informatica – Università di Bologna

Agenda

01 **Group communication**
groups, multicast ordering, membership, virtual synchrony

02 **Publish-subscribe systems**
variants, broker networks, routing

03 **Case studies: streaming platforms**
Kafka, Kinesis, Pulsar, Google Pub/Sub, Redis

04 **Message queues**
IBM MQ, Jakarta Messaging (JMS)

05 **Distributed shared memory**
DSM and tuple spaces

Intersystem communication

Intersystem communication is at the heart of all distributed systems

How can processes on different machines exchange data?

Programming by message passing is harder than using primitives based on shared memory, as in non-distributed platforms.

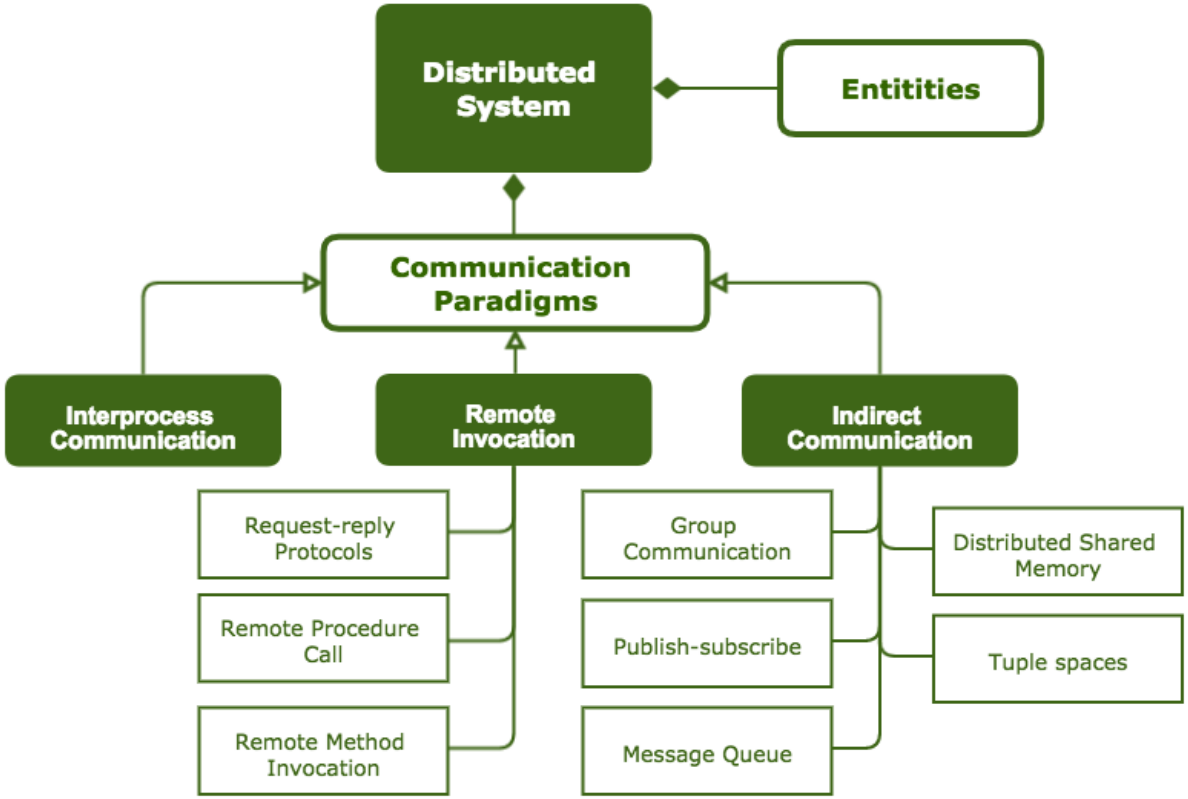
How can a system communicate with multiple systems at the same time?

We need one-to-many abstractions, where the sender does not handle each receiver individually.

What if the receiver is not active when the request is issued?

Asynchronous, *time-uncoupled* communication services are needed.

Communication paradigms in distributed systems



Interprocess communication: low-level primitives, typically built on **sockets**

Remote invocation: request-reply, RPC, RMI

Indirect communication: the subject of today's lecture

Source: dzone.com – Message brokers in the indirect communication paradigm

How can we decouple the sender and the receiver?

Direct communication mechanisms – send/receive messages and remote invocation (RPC/RMI) – introduce a **dependency** between the sender and the receiver, or between the caller and the callee.

Space coupling

The sender must know the identity of the receiver: the two are referentially bound to each other.

Time coupling

Sender and receiver must both exist and be active at the same moment in time.

How can we get rid of this dependency?

Indirect communication

The essence of indirect communication is to communicate **through an intermediary**, with no direct coupling between the sender and the receiver(s). Space and time uncoupling are the key ideas behind:

Group communication

Communication via a group abstraction; the sender is unaware of the identity of the recipients.

Publish-subscribe

Publishers produce events delivered to multiple recipients through an intermediary.

Message queues

Messages are directed to a queue; receivers extract messages from the queue.

Shared-memory approaches

Distributed shared memory and tuple spaces offer the abstraction of a logically shared memory.

Space and time coupling in distributed systems

	Time-coupled	Time-uncoupled
Space-coupled	Communication directed to given receiver(s) that must exist at that moment in time. **Examples: message passing, remote invocation**	Communication directed to given receiver(s); senders and receivers can have independent lifetimes. **Example: open question for the class (Coulouris, Exercise 6.3)**
Space-uncoupled	The sender does not need to know the identity of the receivers; senders and receivers must exist at the same time. **Examples: IP multicast, broadcast**	The sender does not need to know the identity of the receivers; senders and receivers can have independent lifetimes. **Examples: publish-subscribe, message queues, tuple spaces**

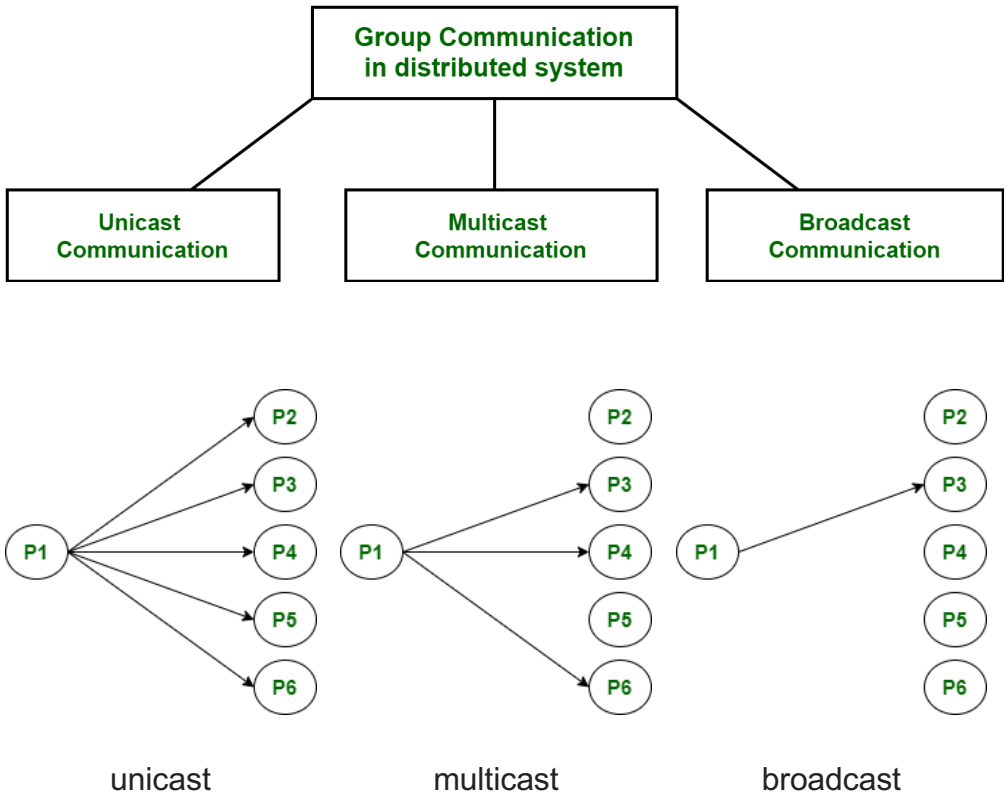
01

Group communication

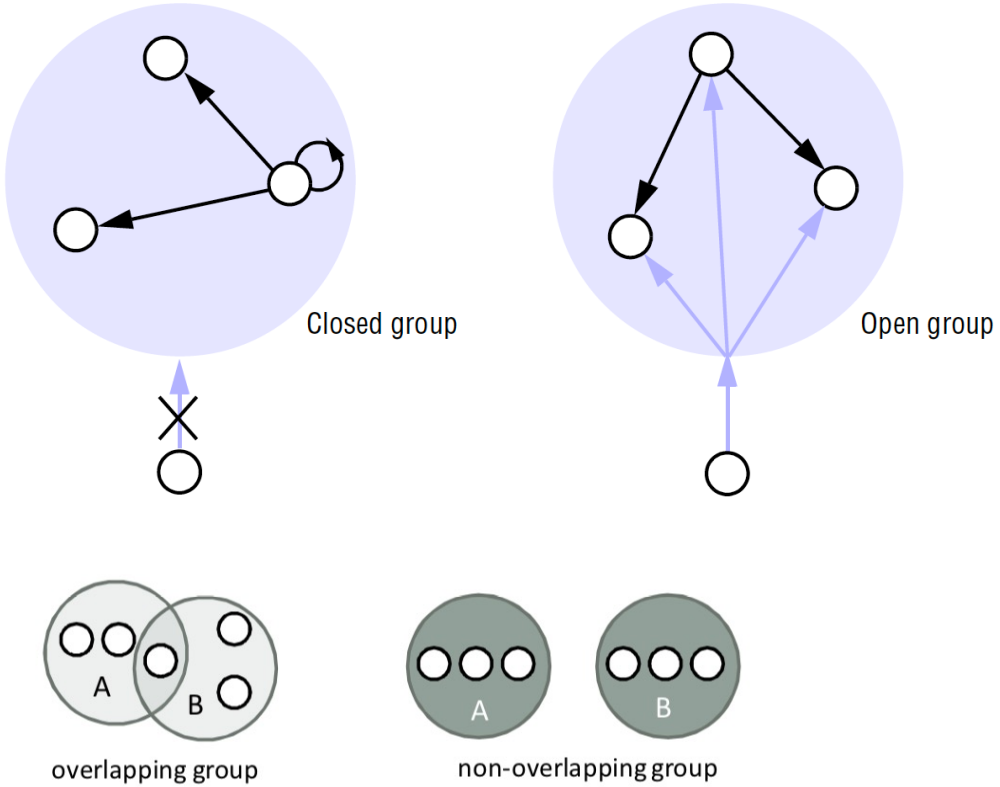
Group communication

When one source process communicates with multiple processes at once, we speak of **group communication**.

- A **group** is a collection of abstractly interconnected processes
- The abstraction hides message passing, so that communication looks like a normal procedure call
- Group communication helps processes on different hosts work together and perform operations in a coordinated way, e.g. to maintain replicated services



Open, closed and overlapping groups



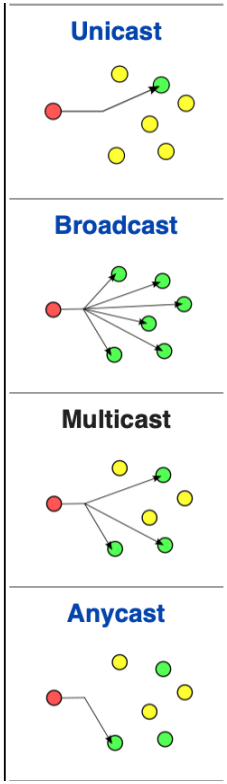
- A group is **closed** if only members of the group may multicast to it; a process in a closed group delivers to itself any message it multicasts to the group
- A group is **open** if processes outside the group may send to it
- In **overlapping groups** a process may be a member of several groups; in non-overlapping groups each process belongs to at most one group
- In real systems, overlapping membership is the realistic case

Multicast

- **Multicast** is group communication where data is addressed to a group of destination computers simultaneously
- It can be one-to-many or many-to-many
- It should not be confused with physical-layer point-to-multipoint communication
- **IP multicast** is a technique for one-to-many communication over an IP network

We use multicast for:

- fault tolerance based on replicated services
- discovering services in spontaneous networking
- better performance through replicated data
- propagation of event notifications



The figure also shows anycast: delivery to any one member of the group

Ordering multicast messages

Ordering is not guaranteed by basic IPC primitives. Group communication services offer **ordered multicast**, with one or more of the following properties (hybrid solutions are possible):

FIFO ordering

Also called source ordering. If a process sends one message before another, they are delivered in this order at all processes in the group.

Causal ordering

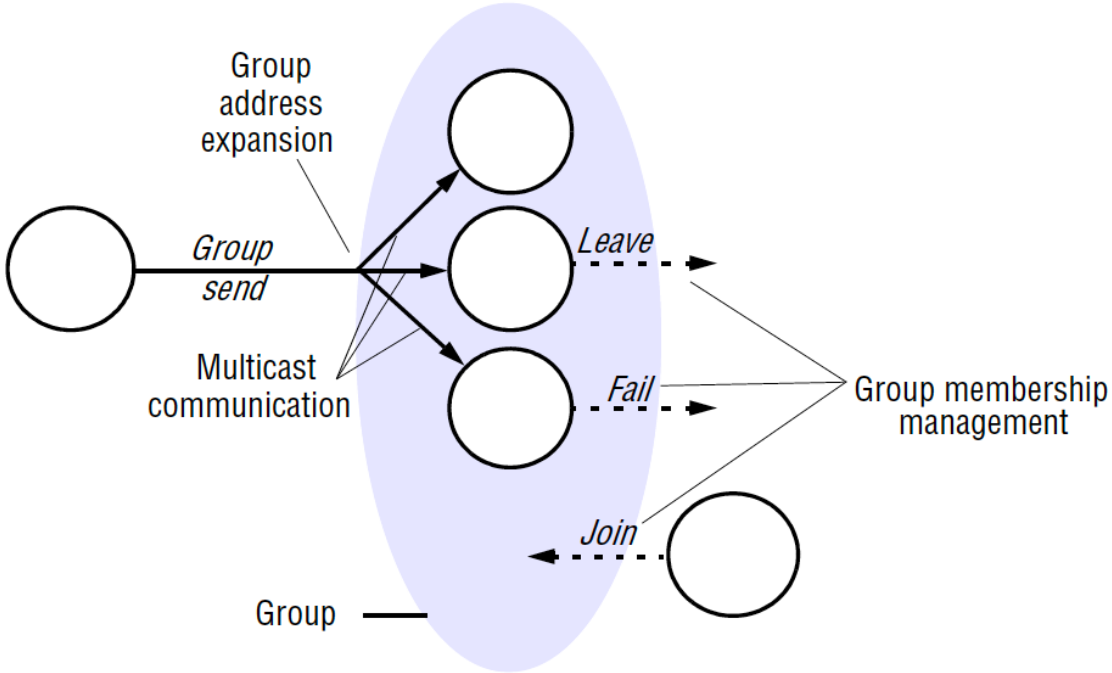
If a message *happens before* another in the distributed system, this causal relationship is preserved in delivery at all processes.

Total ordering

If a message is delivered before another at one process, the same order is preserved at all processes.

Note: total ordering does not imply FIFO or causal ordering – hence hybrids such as FIFO-total and causal-total. We will define *happens before* precisely later.

The role of group membership management



Group membership management keeps an **accurate view** of the current membership, given that entities may **join**, **leave** or even **fail**

The figure shows an *open* group

Group membership service: four main tasks

1. Interface for membership changes

Operations to create and destroy groups and to add or withdraw a process. A process may belong to several groups at the same time (overlapping groups).

2. Failure detection

Monitors members for crashes and for unreachability due to communication failures, marking them *Suspected* or *Unsuspected*; suspected processes are excluded.

3. Notifying membership changes

Notifies members when a process is added or excluded (by failure or deliberate withdrawal).

4. Group address expansion

The sender supplies a group identifier rather than a list of processes; the service expands it into the current membership.

Isis (Ken Birman, Cornell University)

- Birman is the author of the **Isis Toolkit**, which introduced the **virtual synchrony** execution model for multicast communication
- Isis Distributed Systems commercialized it: the software ran the New York and Swiss Stock Exchanges, the French air traffic control system and the US Navy AEGIS warship
- Isis lets distributed systems adapt automatically to failures, share keys and security policies securely, and replicate critical services so they stay available while some components are down
- Birman later released **Vsync** (Isis2) as an open-source library
- More recently his group developed **Derecho** (ACM TOCS 2019), a high-performance successor designed for RDMA data-centre networks

The virtual synchrony model

- Virtual synchrony is an ordered, reliable multicast technology for interprocess message passing
- Programs organize themselves into **process groups** and send messages to groups rather than to specific processes
- Group membership evolves through a sequence of **views**; view changes are delivered in a consistent order with respect to messages
- **Key property:** processes that survive from one view to the next deliver the *same set* of messages in that view – every message is delivered either to all surviving members or to none
- Combined with ordered delivery, every member sees the same events (membership changes and messages) in the same order: this greatly simplifies application design

Implementing the virtual synchrony model

A virtually synchronous service is typically implemented with **state machine replication**:

1 Write one program

A single program receives inputs from clients via message passing, enters a new state and responds deterministically.

2 Replicate it

Multiple instances run on different machines; a virtually synchronous system replicates incoming messages to all members.

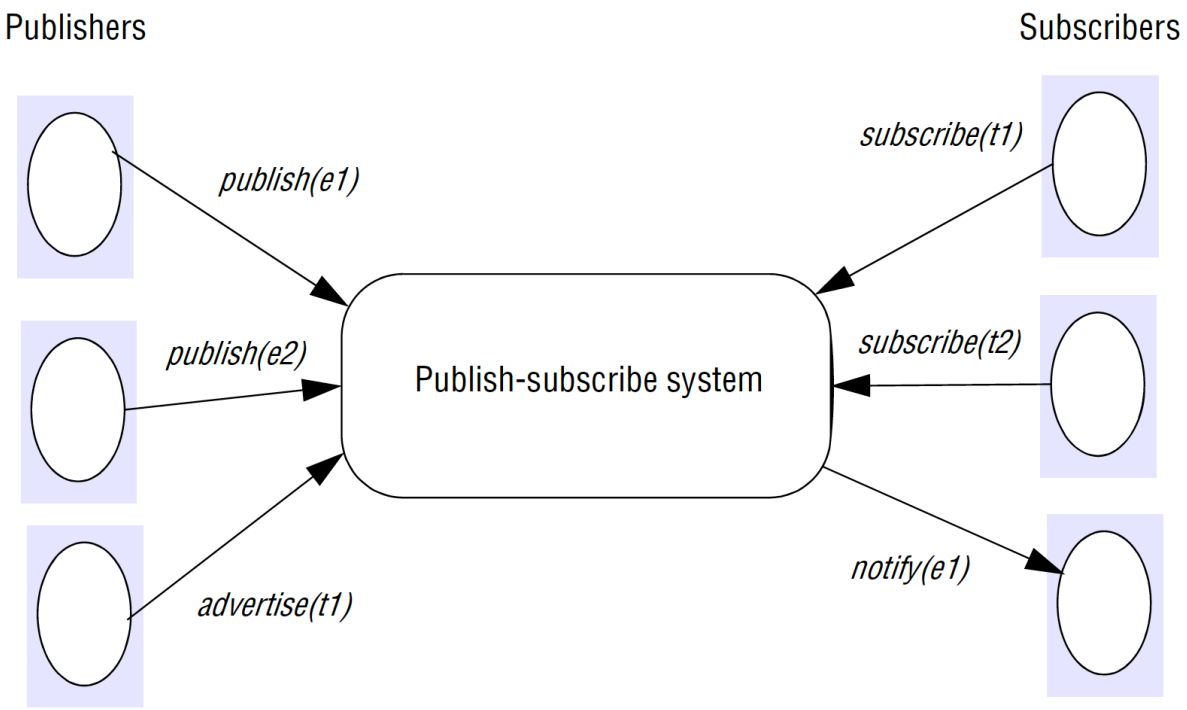
3 Stay consistent

Replicas see the same events in the same order, make the same state transitions and remain in a consistent state.

02

Publish-subscribe systems

The publish-subscribe architectural pattern



Publish/subscribe is an interaction pattern: subscribers express interest in messages; publishers produce messages without specifying the recipients

The exchange is **time-decoupled and anonymous**

publish(e)	**subscribe(f)**
unsubscribe(f)	**notify(e)**
advertise(f)	

Publish-subscribe systems: main features

Heterogeneity

Software components designed independently can work together.

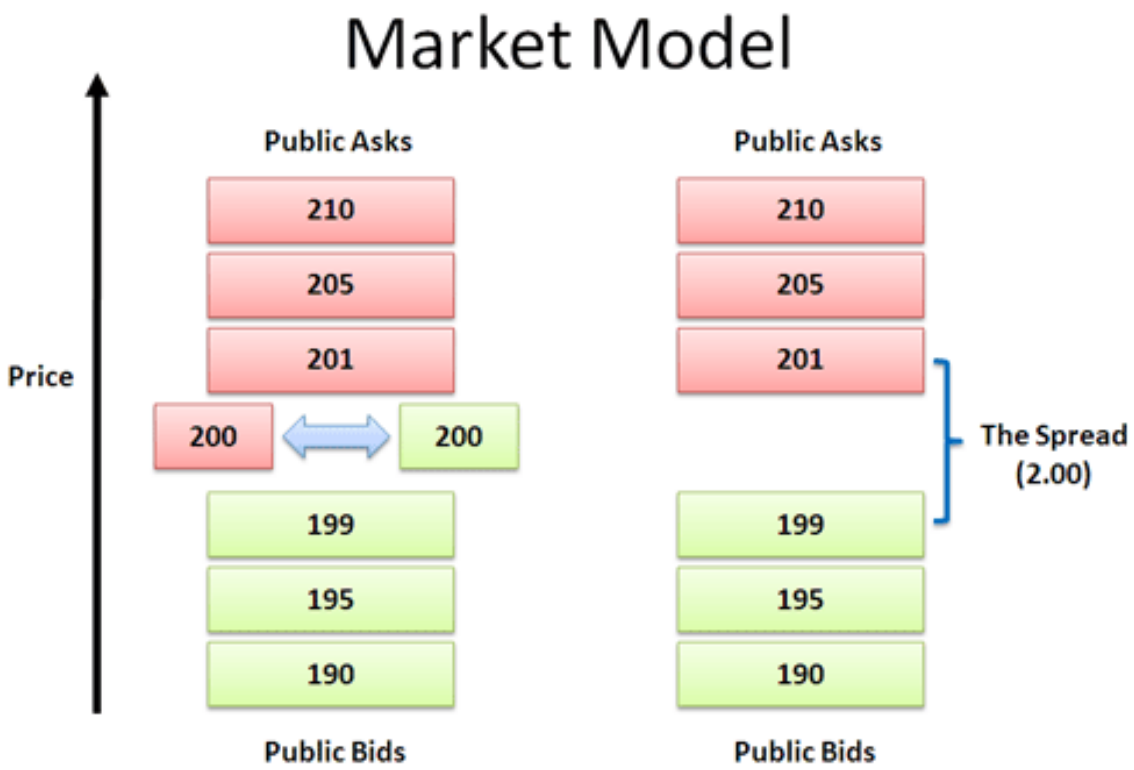
Asynchronicity

Publishers and subscribers are time-decoupled.

Delivery guarantees

A variety of guarantees can be provided for notifications; the choice depends on the application requirements.

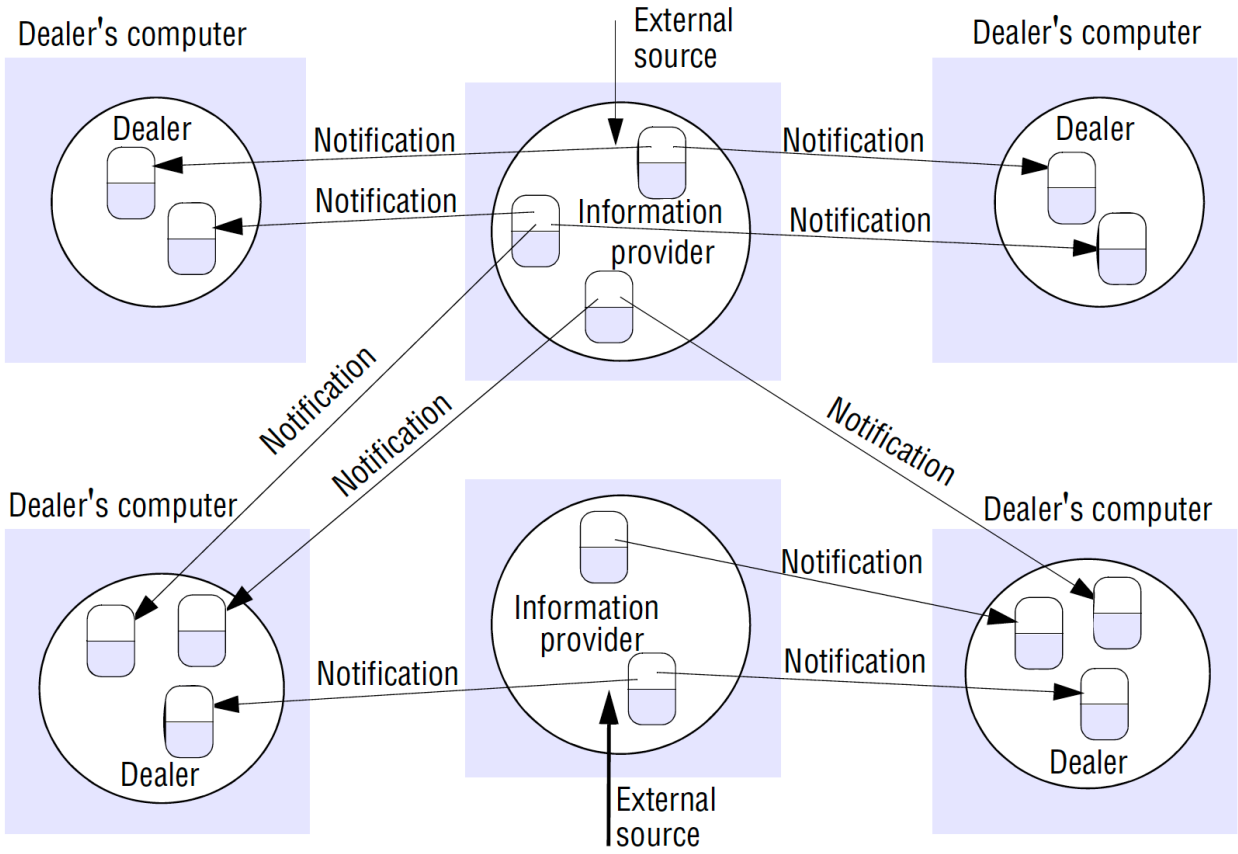
Stock exchange: an example of distributed system



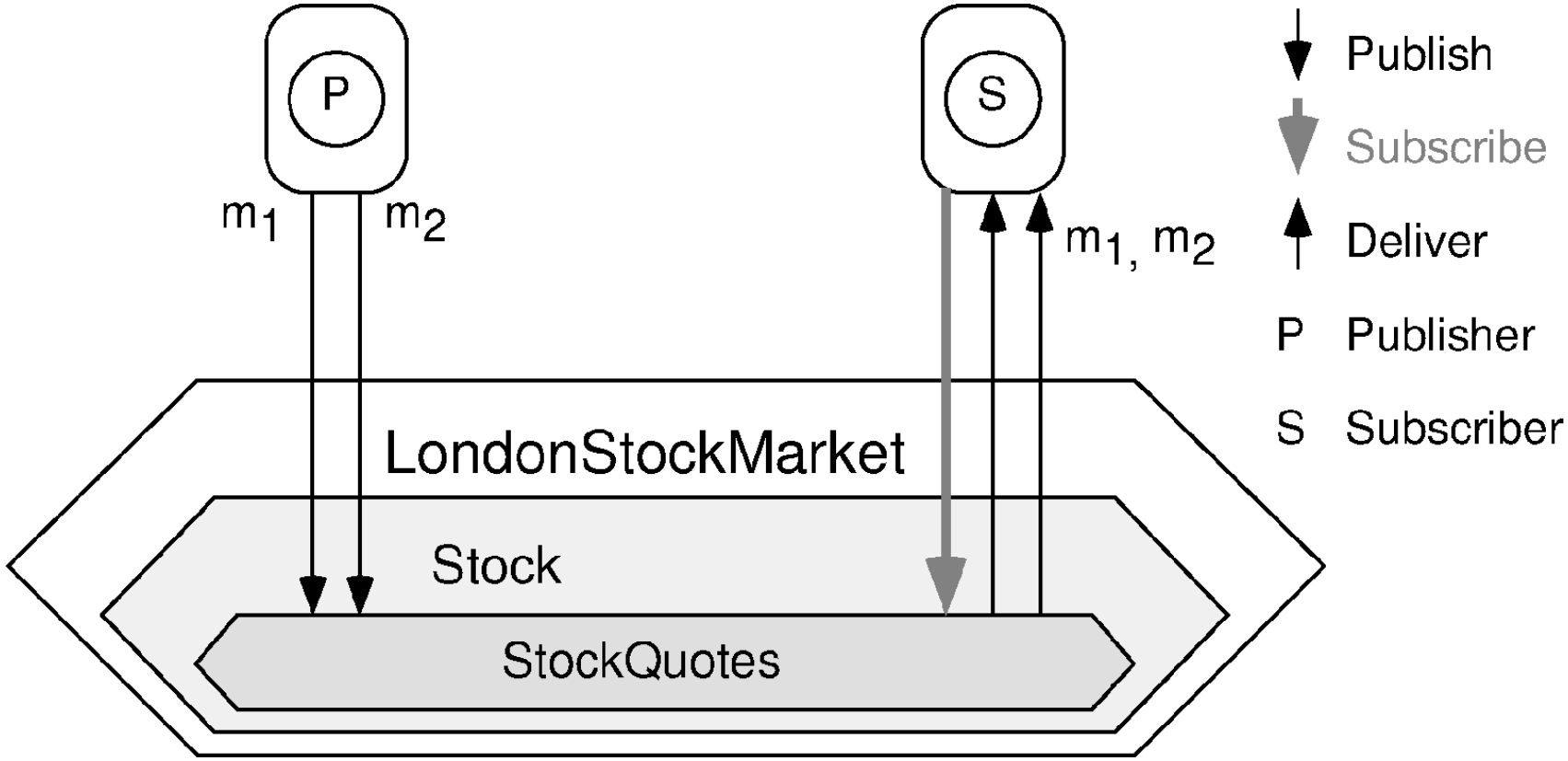
The order book: bids, asks and spread

Dealing room system

- A simple dealing room lets dealers see the latest market prices of the stocks they deal in
- The market price of a single named stock is represented by an associated object
- Information arrives from several external sources as updates (events) to some or all of these objects



Example: stock exchange as publish/subscribe



From: Eugster et al., The many faces of Publish/Subscribe, ACM Computing Surveys 35(2), 2003

Variants of publish-subscribe: channel-based

Channel-based

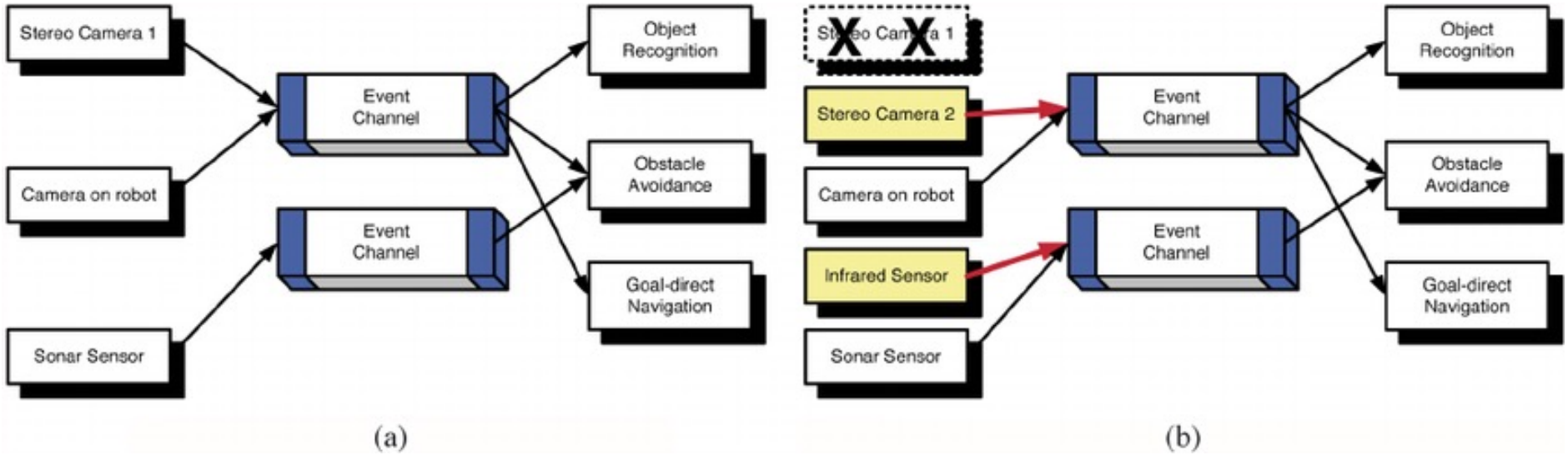
Publishers publish events to *named channels*; subscribers subscribe to a named channel and receive *all* events sent to that channel.

Example: the CORBA Event Service

Example: CORBA Event Service

Robot navigation using an event channel

When the configuration changes, we just reconfigure the event channel connections: e.g. (a) → (b) adds a stereo camera and an infrared sensor and removes stereo camera 1



Source: Song, CONCORD: A Control Framework for Distributed Real-Time Systems, IEEE Sensors Journal, 2007

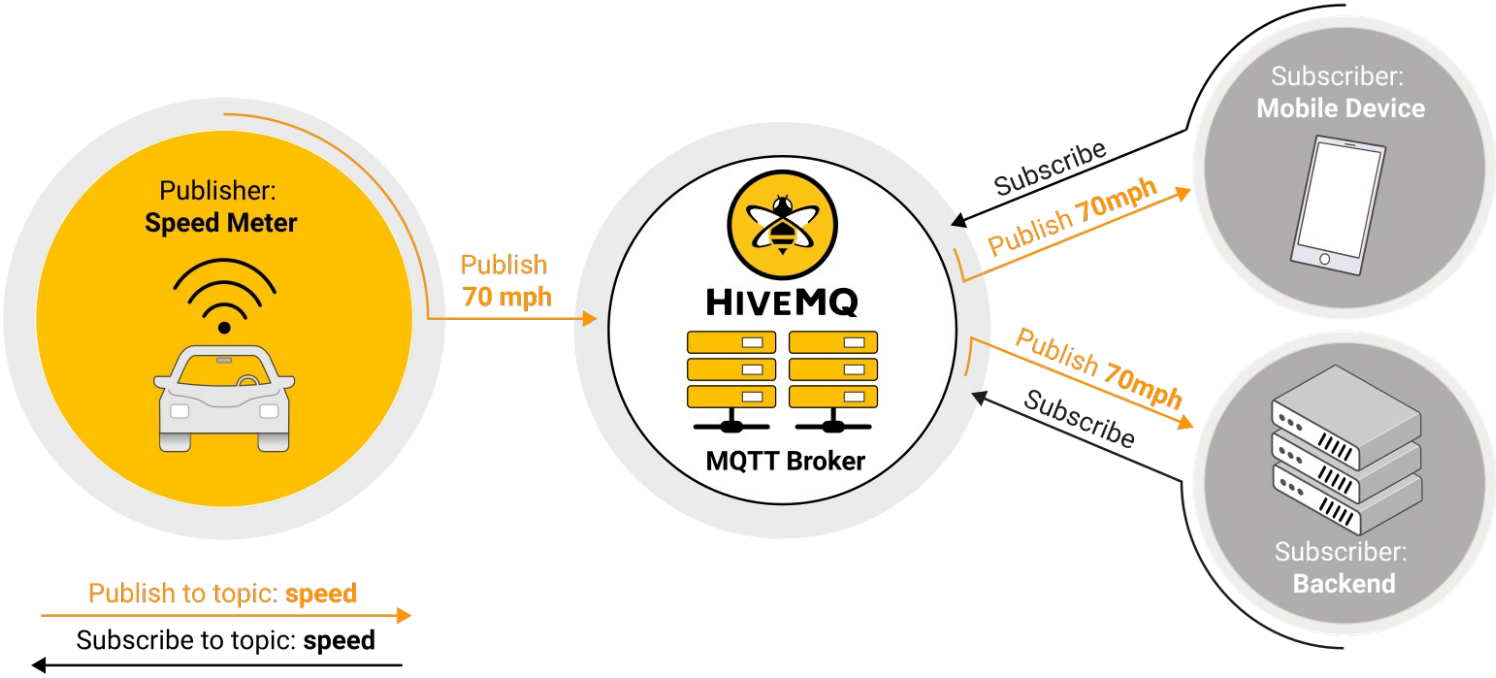
Variants of publish-subscribe: topic-based

Topic-based (also subject-based)

- Each notification is expressed in terms of a number of *fields*, one of which denotes the **topic**
- Subscriptions are defined in terms of the topic of interest

Equivalent to channel-based approaches, except that topics are *implicitly* defined by channels, while they are *explicitly* declared as a field in topic-based approaches.

Example: topic-based publish-subscribe in IoT with MQTT



https://www.hivemq.com/blog/how-to-get-started-with-mqtt/

Variants of publish-subscribe: content-based

Content-based

A generalization of topic-based approaches: subscriptions can be expressed over a *range of fields* in an event notification.

- A content-based filter is a query built from compositions of constraints over the values of event attributes
- Example: events about sport and regional news, where the sport is **soccer** and the country is **Italy**
- Query languages vary from system to system, but this approach is significantly more expressive than channel- or topic-based ones

Example: a content-based publish-subscribe system

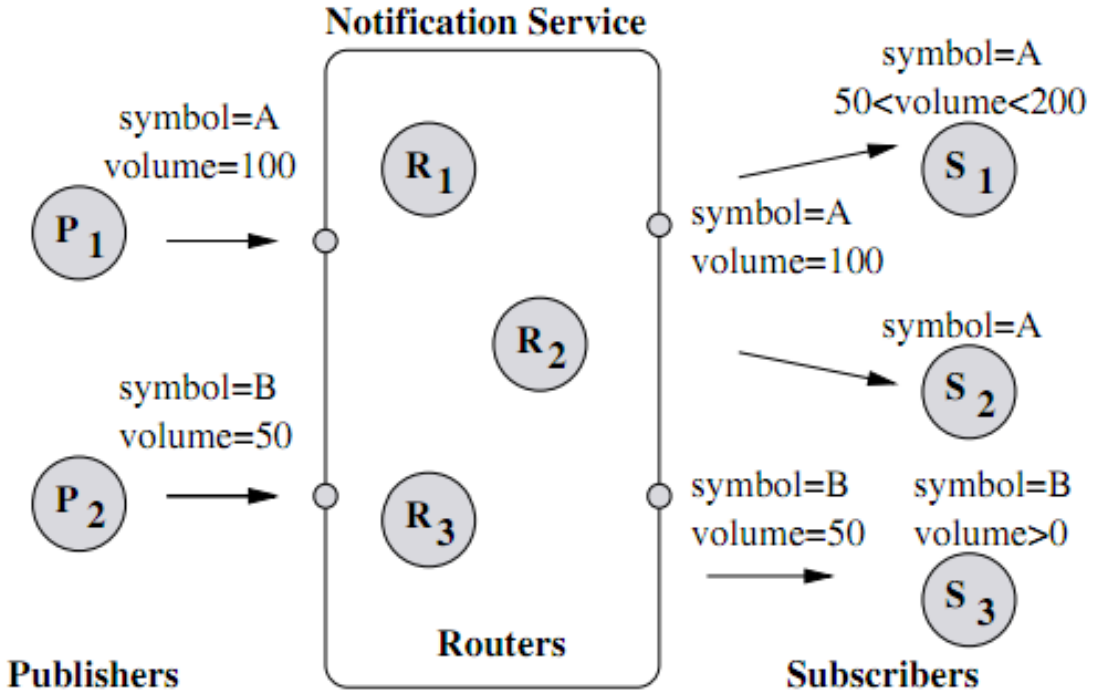


Figure 4.6: Example of content-based publish/subscribe.

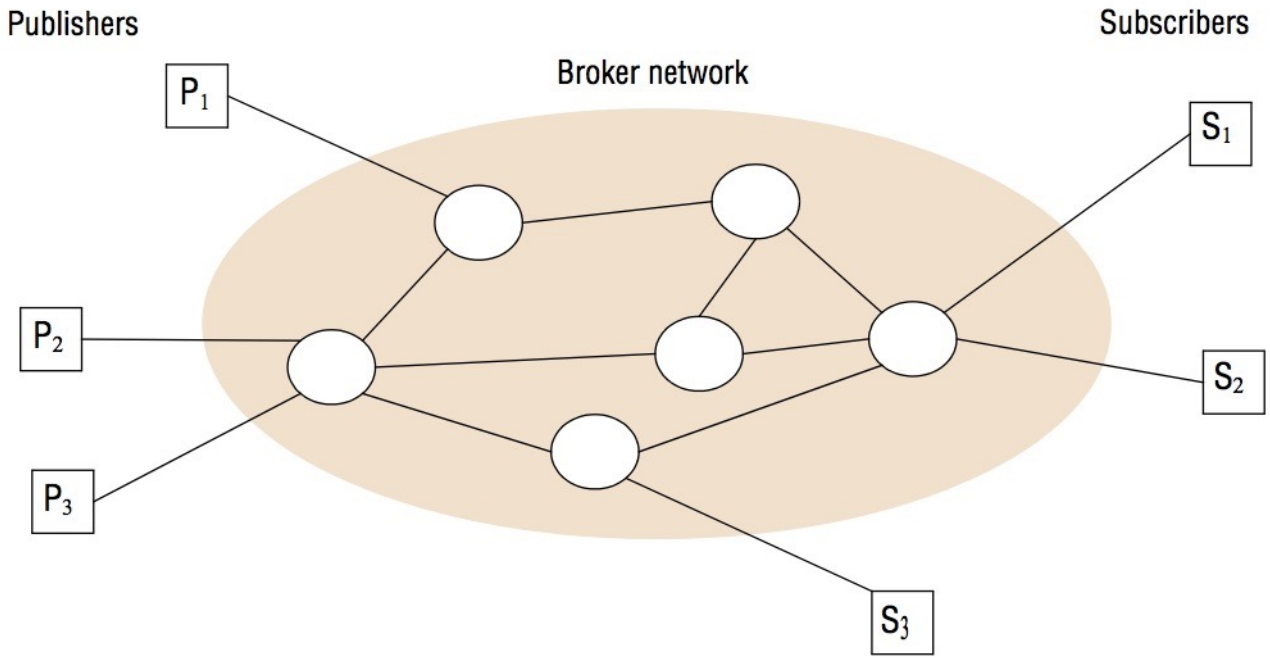
Variants of publish-subscribe: type-based

Type-based

Linked to object-based approaches where objects have a specified type: subscriptions are defined in terms of *types* of events, and matching in terms of types or subtypes of the filter.

- Filters range from coarse-grained (overall type names) to fine-grained queries on attributes and methods of an object – the latter are similar to content-based approaches
- **Advantages:** elegant integration into programming languages, and type checking of subscriptions, which eliminates some kinds of subscription errors

A network of brokers



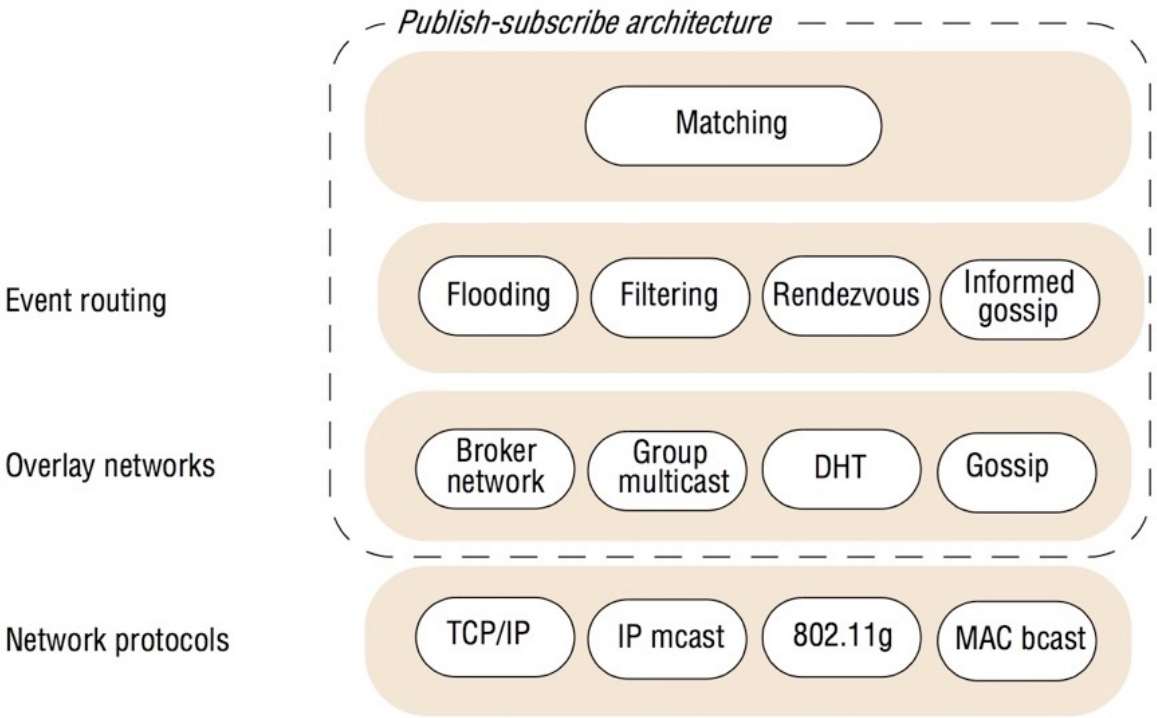
The simplest implementation is *centralized*: a single server acts as the event broker

Easy to build, but the broker is a **single point of failure** and a **performance bottleneck**

Distributed implementations replace it with a **network of cooperating brokers**

Layered architecture of publish-subscribe systems

- **Matching** (top layer) ensures events match subscriptions; often pushed down into event routing
- **Event routing** delivers notifications as efficiently as possible to the right subscribers
- **Overlay networks** support routing with networks of brokers or peer-to-peer structures
- **Network protocols** at the bottom: TCP/IP, IP multicast, wireless



Implementations of publish-subscribe

Flooding

Send each event notification to *all* nodes; matching is done at the subscriber end.

Filtering

Brokers forward notifications only along paths leading to *valid* subscribers.

Advertisements

Publishers *advertise* the events they will publish, reducing subscription traffic.

Rendezvous

Responsibility for the *event space* is partitioned among the brokers.

03

Case studies: streaming platforms

Some modern publish-subscribe systems

Apache Kafka

Distributed, partitioned and replicated commit log

Apache Pulsar

Brokers separated from storage (Apache BookKeeper)

Amazon Kinesis

Managed streaming services on AWS

Google Cloud Pub/Sub

Global, fully managed messaging service

Azure Event Hubs

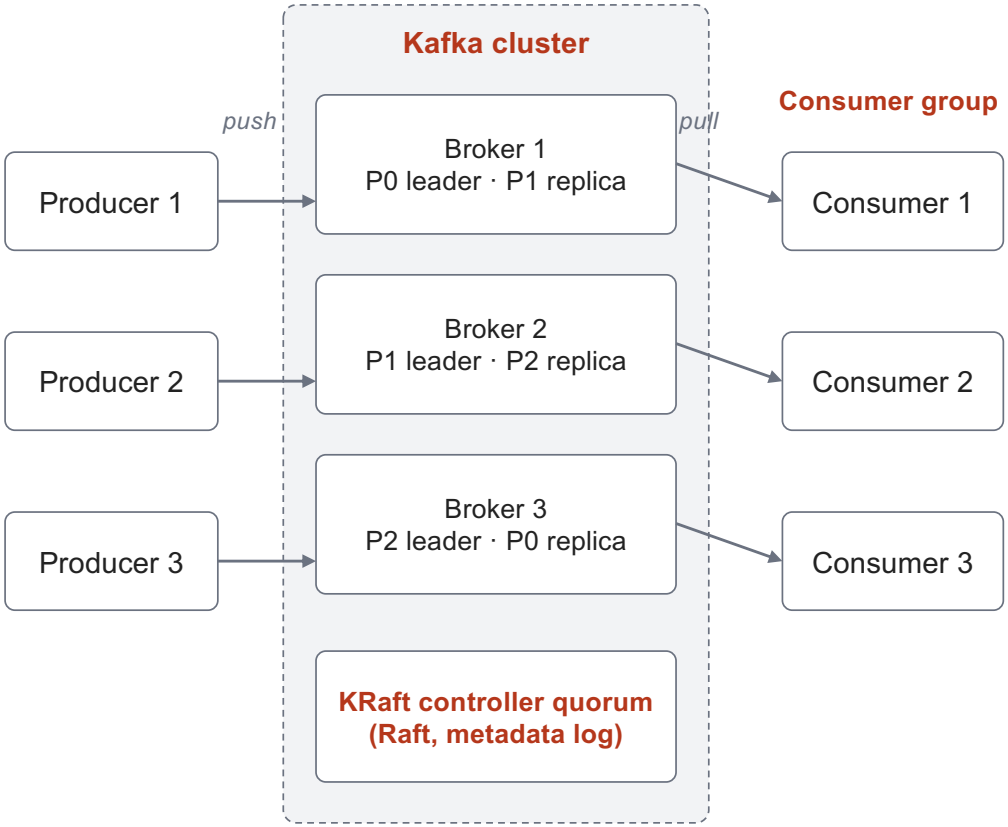
Managed event ingestion, Kafka-compatible endpoint

Redis

Pub/Sub channels and Redis Streams

Apache Kafka

- Platform for real-time data feeds, written in Java and Scala
- Topics are split into **partitions**, replicated across multiple brokers for load balancing and fault tolerance
- Brokers are **stateful**: they store partition logs on disk; consumers track their position through *offsets*
- Cluster metadata is managed by **KRaft**, a Raft-based controller quorum – production-ready since Kafka 3.3
- **Kafka 4.0 (2025) removed ZooKeeper entirely**



Kafka use cases: LinkedIn and Netflix

Apache Kafka at Large Scale → No need to do a POC

LinkedIn (2018)

Operation Challenges

- The scale of Kafka deployment @LinkedIn
  - 2,100+ brokers
  - ~ 60,000 topics
  - > 4.5 trillion messages / day
- Huge operation overhead
  - Hardware failures are norm
  - Workload skews

Strata DATA CONFERENCE

NETFLIX (2018)

Kafka @ Netflix Scale

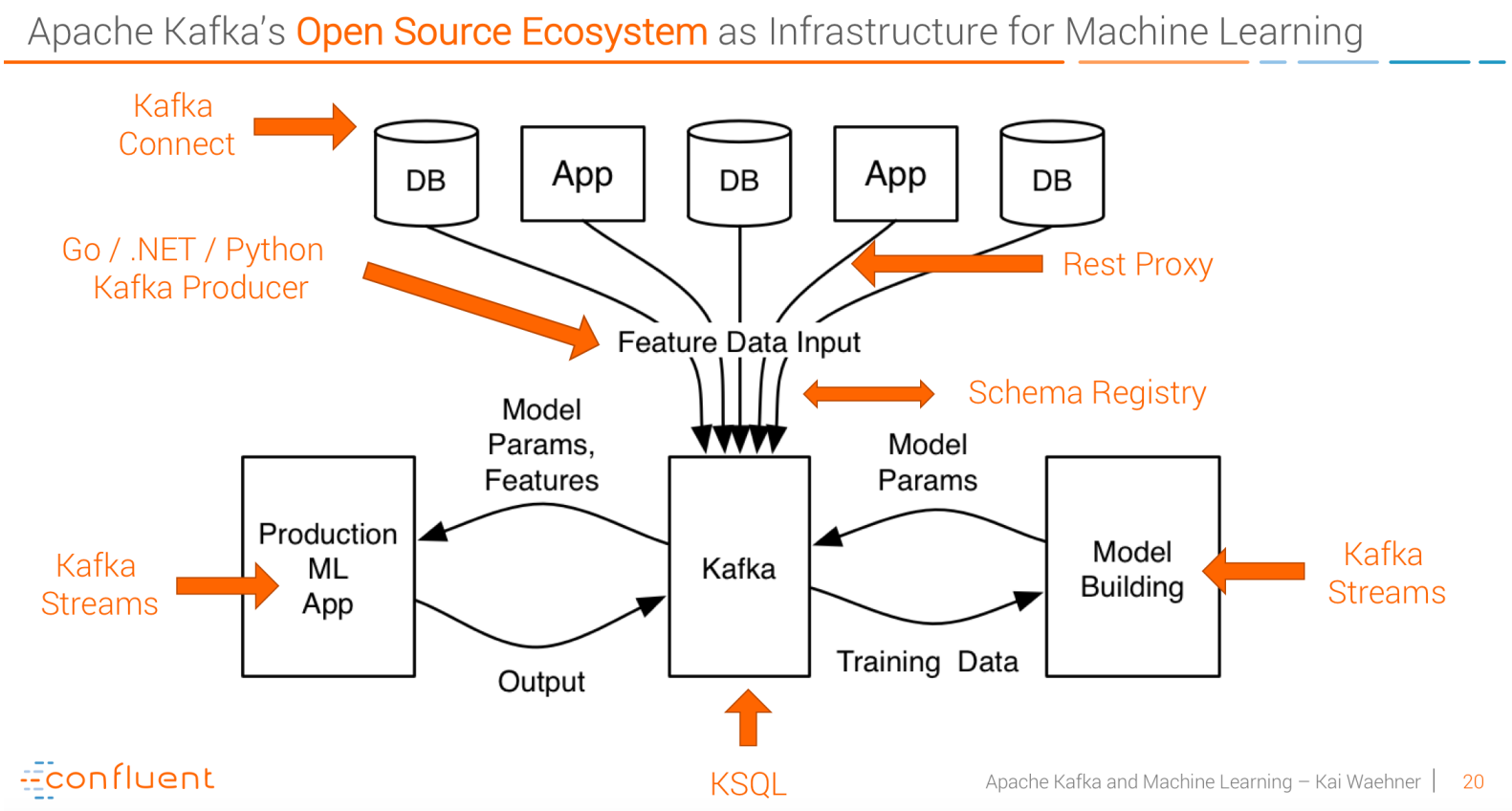
- 4,000+ brokers and ~50 clusters in 3 AWS regions
- > 1 Trillion messages per day
- At peak (New Years Day 2018)
  - 2.2 trillion messages (1.3 trillion unique)
  - 6 Petabytes

QCon

https://conferences.oreilly.com/strata/strata-ca/public/schedule/detail/63921
https://qconlondon.com/london2018/presentation/cloud-native-and-scalable-kafka-architecture

Figures from 2018 conference talks (Strata, QCon); current deployments are considerably larger

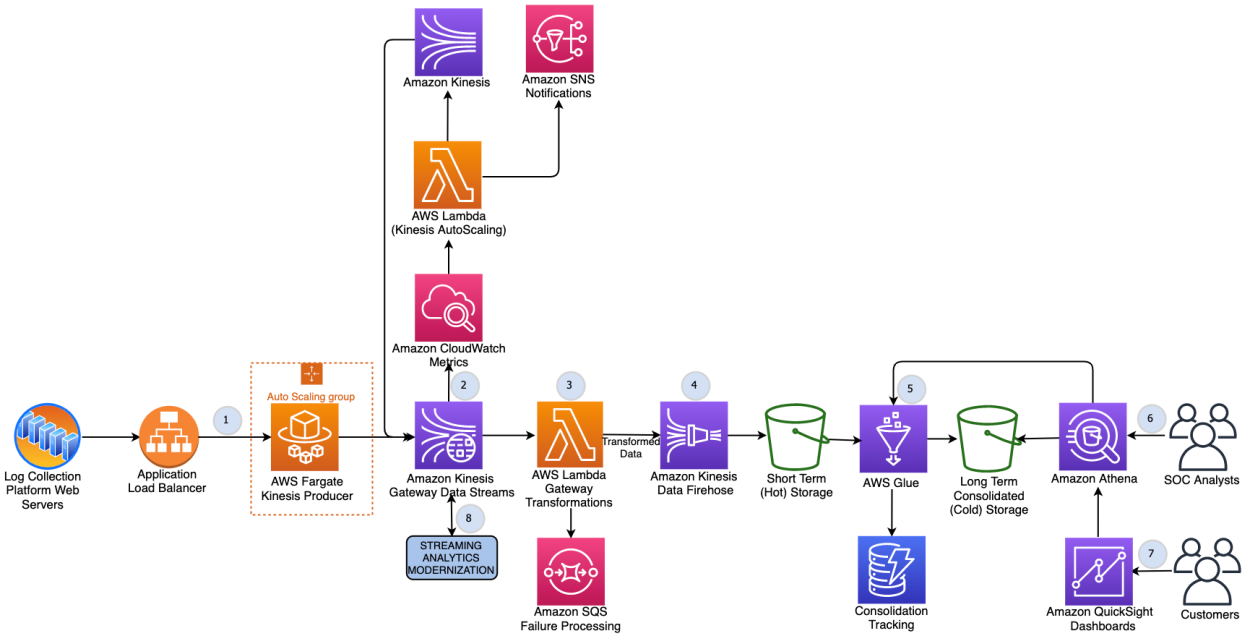
Kafka use cases: an architecture including machine learning



Amazon Kinesis

A family of AWS services for processing and analysing real-time streaming data at large scale:

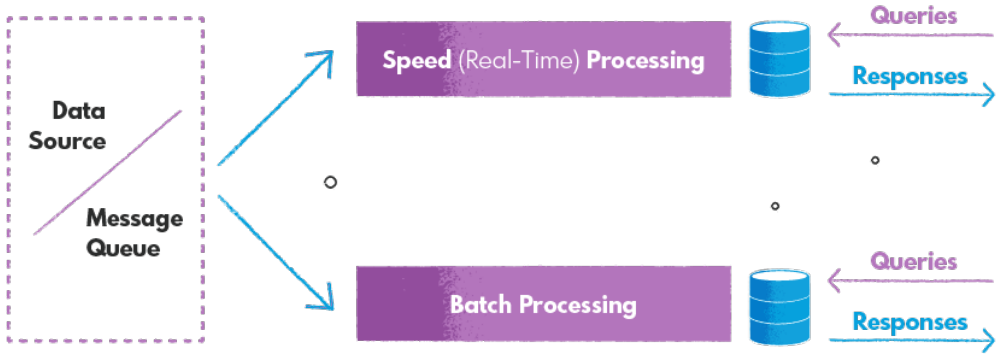
- **Kinesis Data Streams**
- **Kinesis Video Streams**
- **Amazon Data Firehose** (formerly Kinesis Data Firehose)
- **Amazon Managed Service for Apache Flink** (formerly Kinesis Data Analytics)



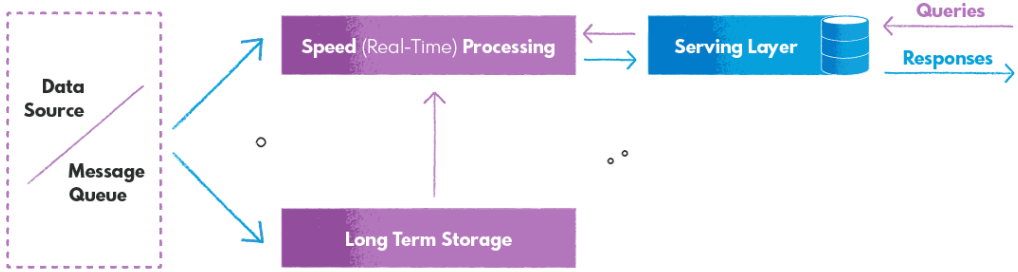
aws.amazon.com – Designing a high-volume streaming data ingestion platform natively on AWS

Stream data processing: Lambda and Kappa architectures

Lambda



Kappa



Data processing architectures help businesses analyse and extract insights from their data in real time

- **Lambda** uses separate batch and stream processing systems: scalable and fault-tolerant, but complex to set up and maintain, as it duplicates processing logic
- **Kappa** treats all data as streams with a single stream processing system: simpler and easier to maintain, but still requires experience in stream processing and distributed systems

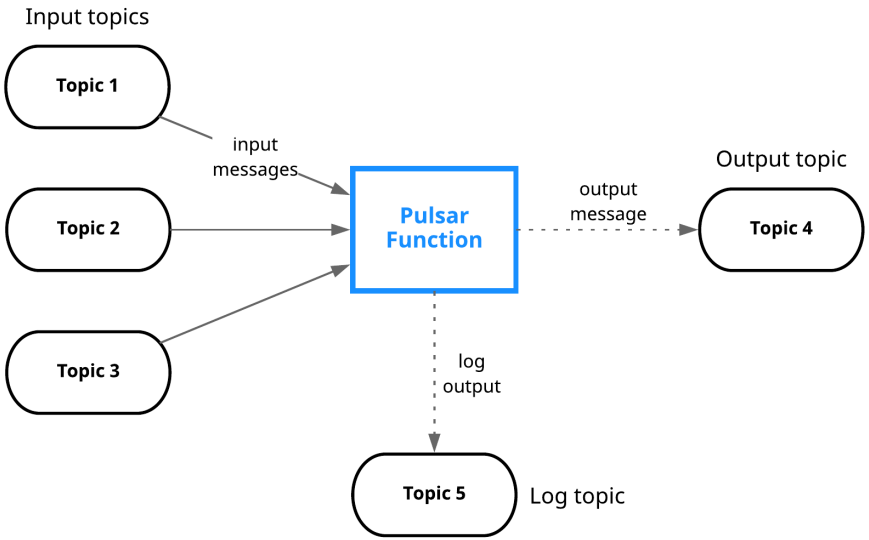
nexocode.com – Lambda vs Kappa architecture

Apache Pulsar

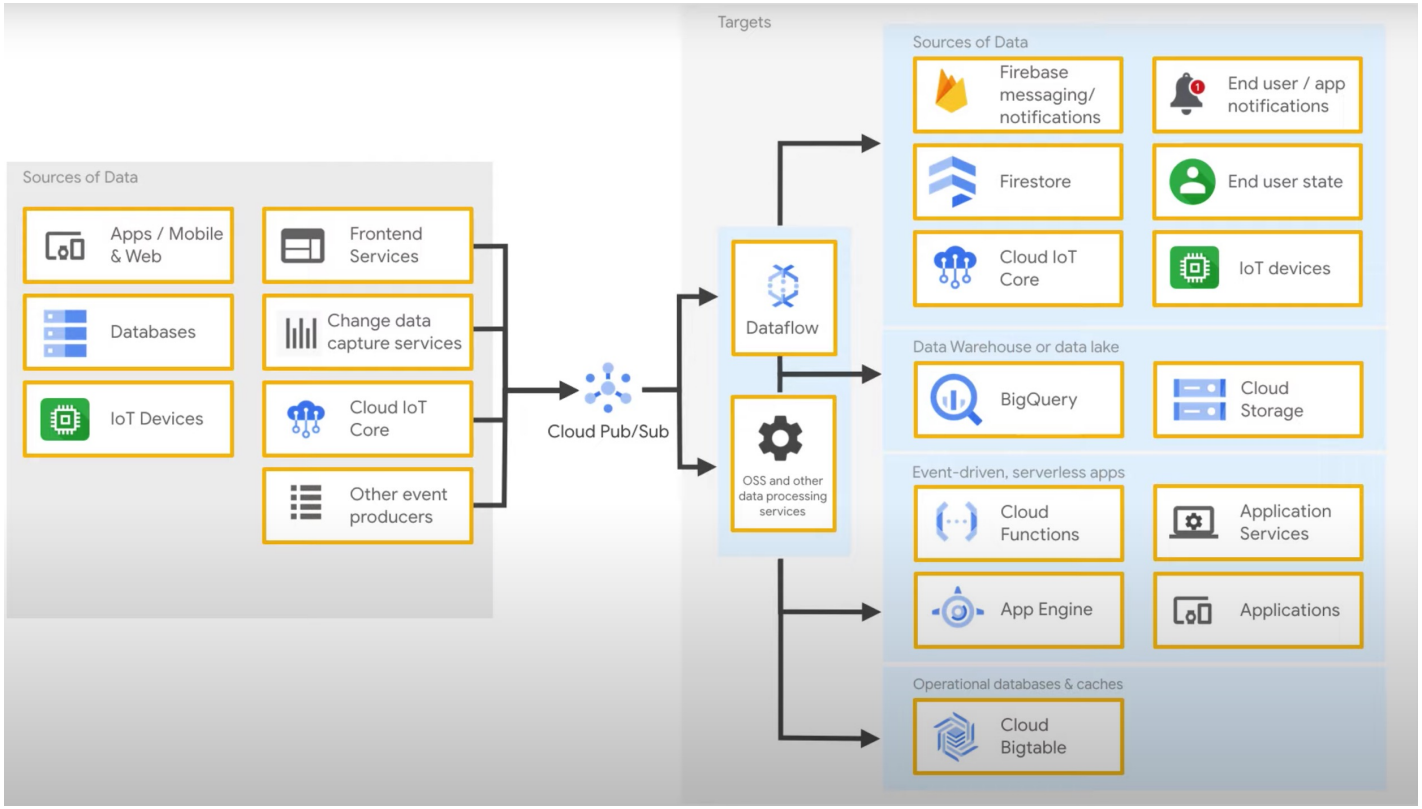
- Cloud-native, distributed messaging and streaming platform created at Yahoo!, now a top-level Apache Software Foundation project

Pulsar Functions are inspired by:

- stream processing engines: Apache Storm, Heron and Flink
- serverless / Function-as-a-Service platforms: AWS Lambda, Google Cloud Functions, Azure Functions



Google Cloud Pub/Sub

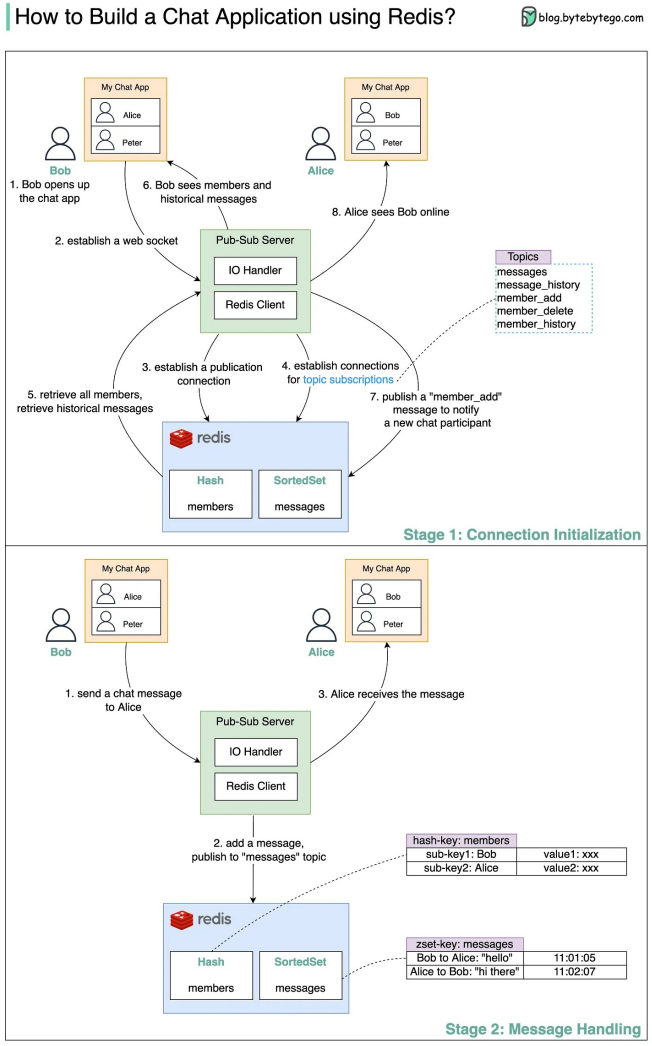


cloud.google.com/pubsub/docs/pubsub-basics – Note: the figure predates the retirement of Cloud IoT Core (August 2023)

Redis

- **Pub/Sub channels**: fire-and-forget – messages are not persisted, and subscribers that are offline miss them
- **Redis Streams**: an append-only log with consumer groups and acknowledgements, closer to Kafka

The figure shows a chat application built on Redis



04

Message queues

Message queues

A simple programming model: **producers** send messages to a specific queue; **consumers** receive messages from that queue. Three styles of receive are generally supported:

Blocking receive

Blocks until an appropriate message is available.

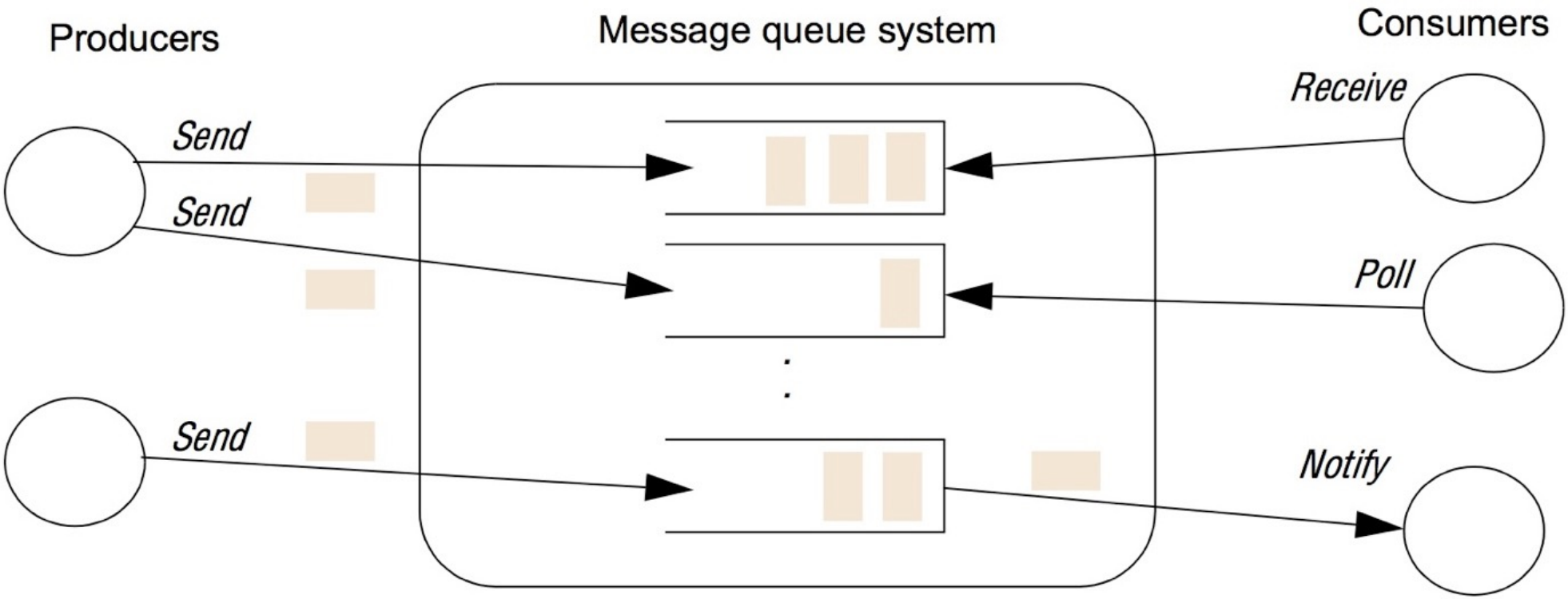
Non-blocking receive

Polling: returns a message if available, or a *not available* indication otherwise.

Notify

Issues an event notification when a message is available in the queue.

The message queue paradigm

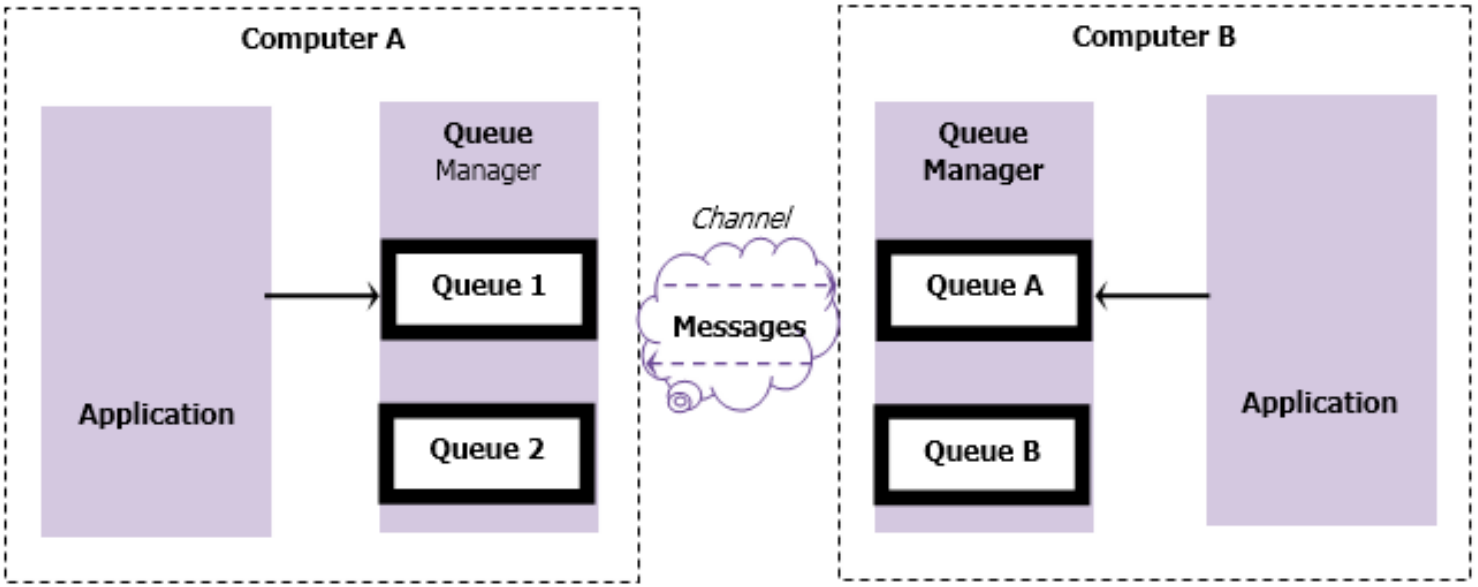


IBM MQ (formerly WebSphere MQ)

- Middleware by IBM based on message queues, offering an **indirection** between senders and receivers
- Client applications accessing a queue manager may reside on the same physical server
- More generally they run on different machines and communicate with the queue manager through a **client channel**
- Client channels adopt the familiar concept of a *proxy*

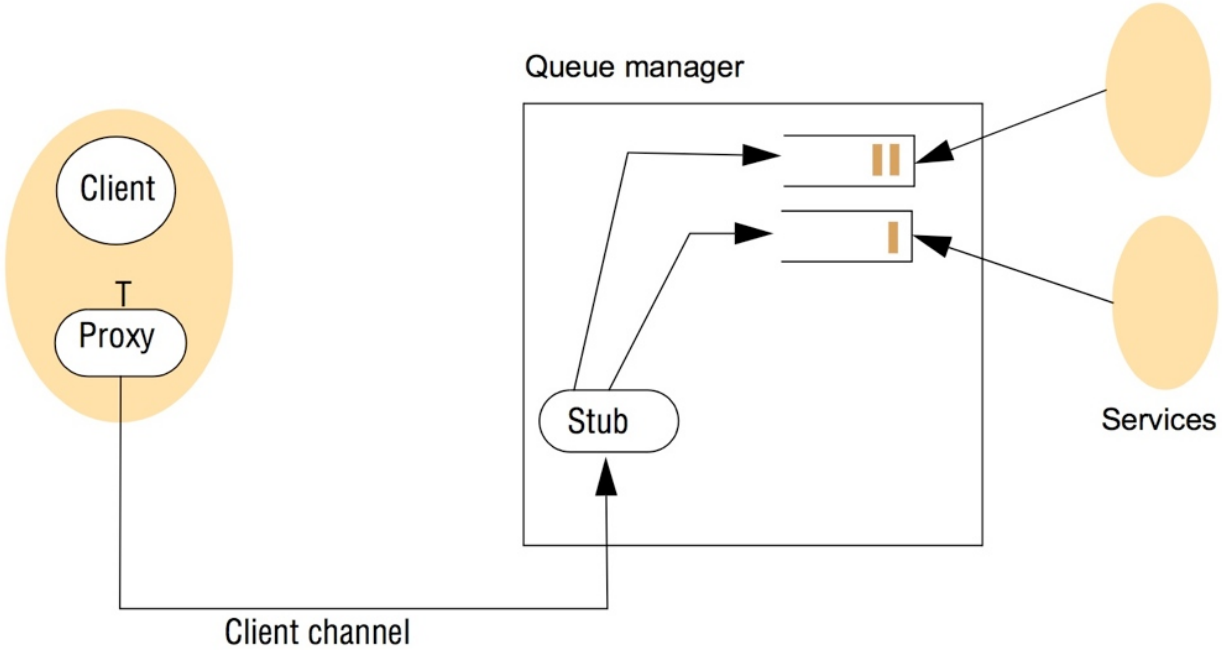
IBM MQ documentation: https://www.ibm.com/docs/en/ibm-mq

IBM MQ architecture



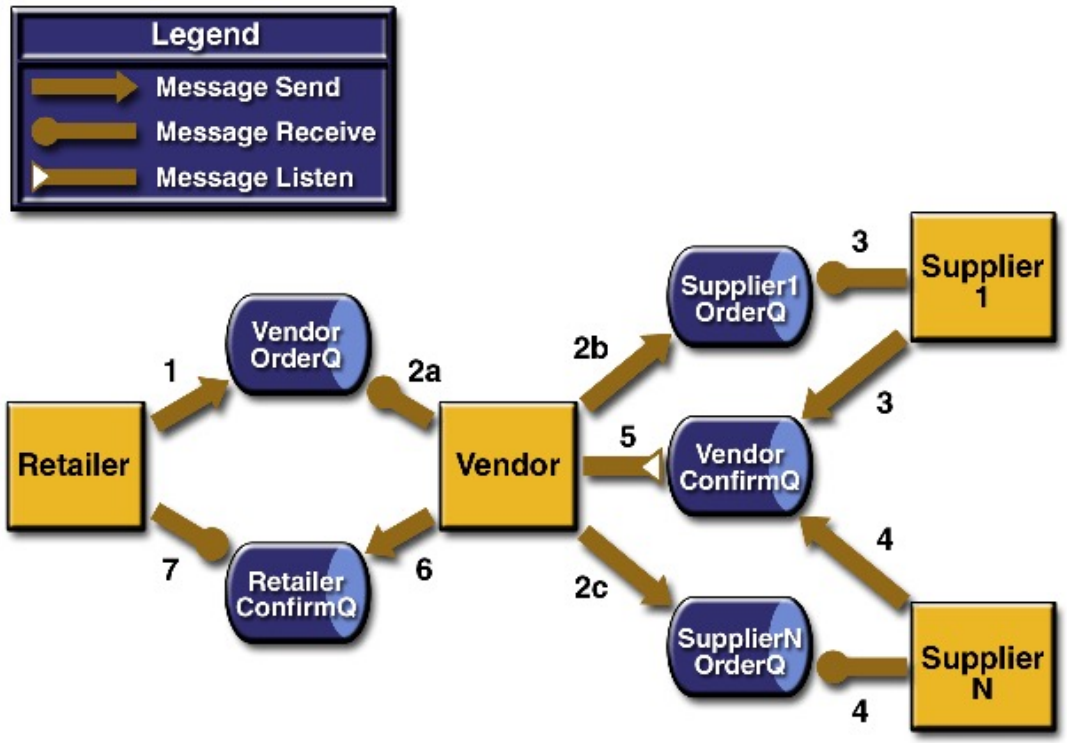
- A **queue manager** provides messaging services to applications and routes messages to the correct queue or to another queue manager
- A **queue** is a container for messages; **channels** are unidirectional or bidirectional

A simple networked topology in IBM MQ



A client application sends messages to a remote queue manager through a client channel (proxy/stub); multiple services on the server machine consume the incoming messages

Jakarta Messaging (formerly Java Message Service, JMS)

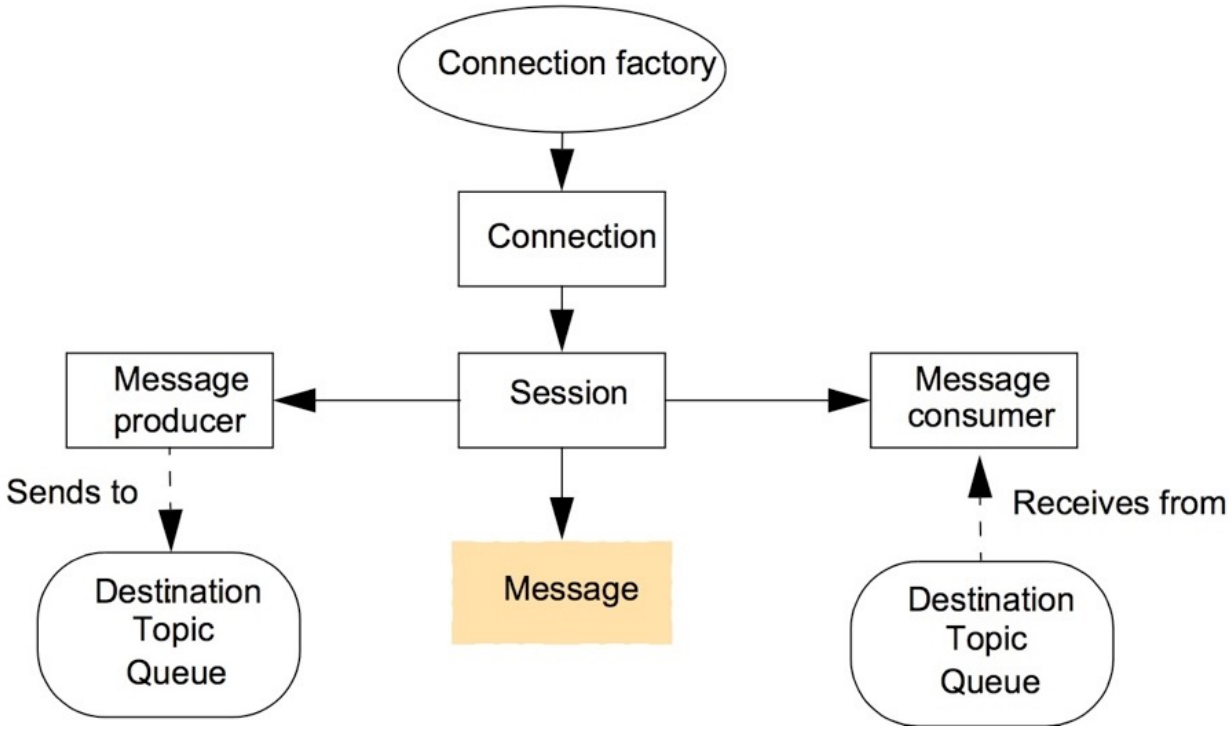


A retailer–vendor–suppliers scenario:

1. The retailer sends an order to the vendor queue
2. In one transaction, the vendor receives it and forwards orders to the suppliers' queues
3. Each supplier ships the items to the destination in the *JMSReplyTo* field
4. An asynchronous listener at the vendor collects the confirmations
5. The vendor notifies the retailer whether the order can be fulfilled

Adapted from the Java Message Service tutorial – client samples

The programming model of Jakarta Messaging (JMS)

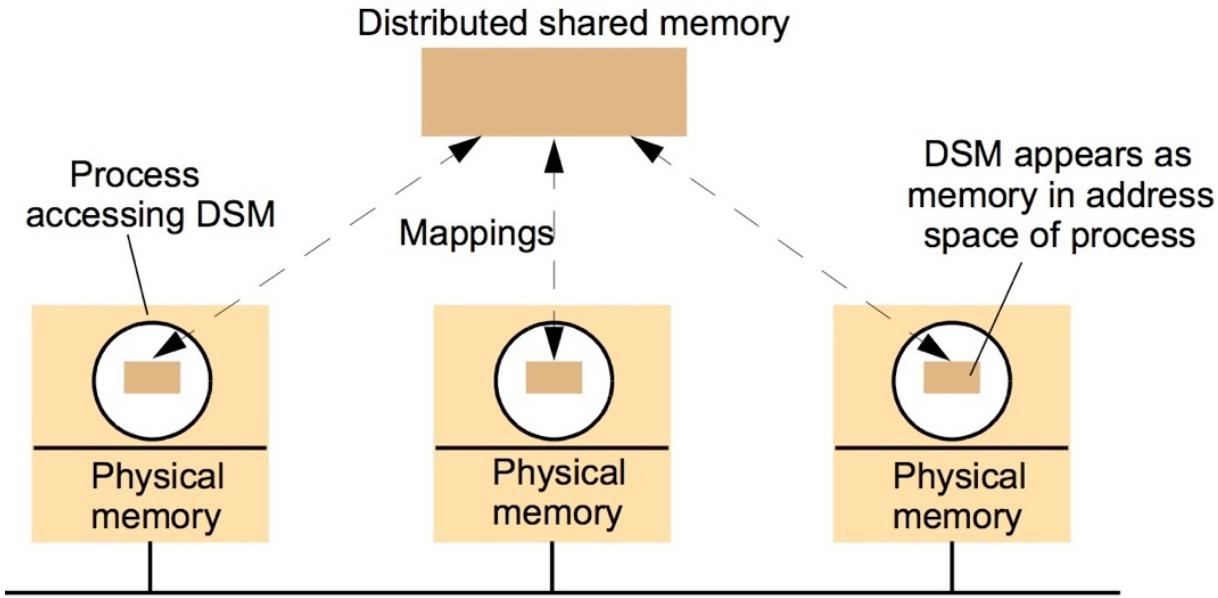


- A **connection factory** creates connections with the required properties
- A **connection** is a logical channel between client and provider (e.g. over TCP/IP)
- Sessions create **producers**, **consumers** and **messages** for a topic or a queue destination

05

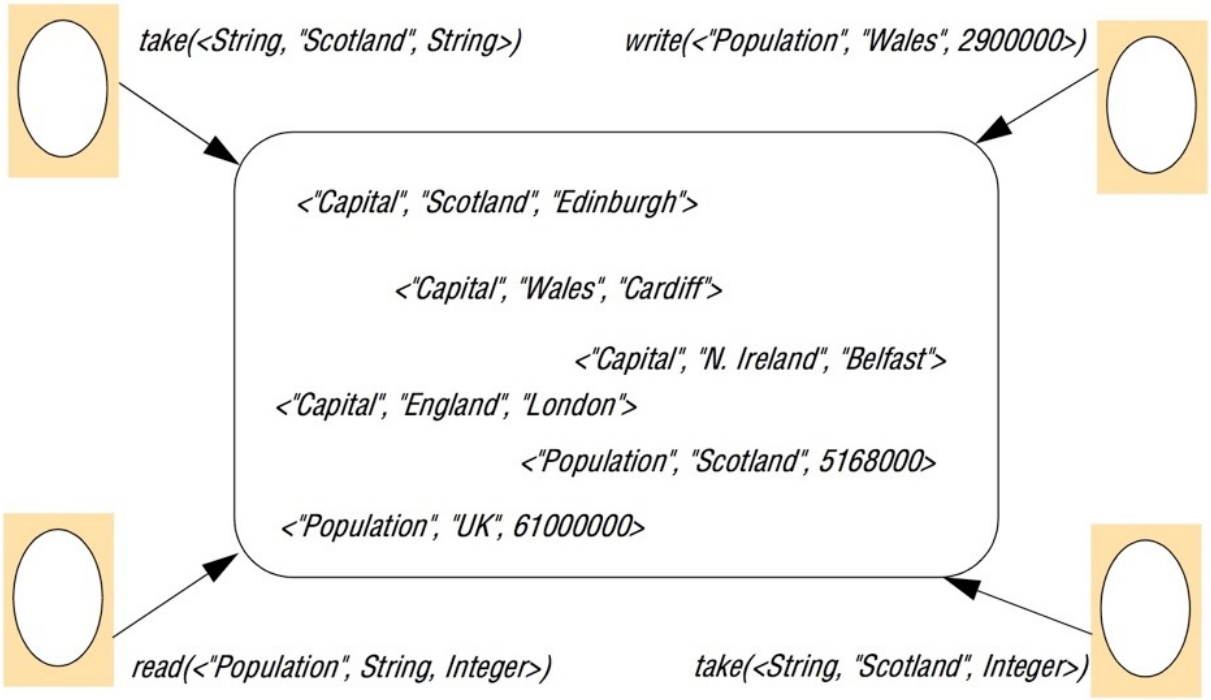
Distributed shared memory

The distributed shared memory (DSM) abstraction



- Processes access DSM through ordinary reads and updates within their own address space
- A runtime system ensures **transparently** that processes on different computers observe each other's updates
- **Key design issues:** consistency model, granularity of sharing (page or object), false sharing and thrashing
- Mainly used for parallel computing on clusters

The tuple space abstraction



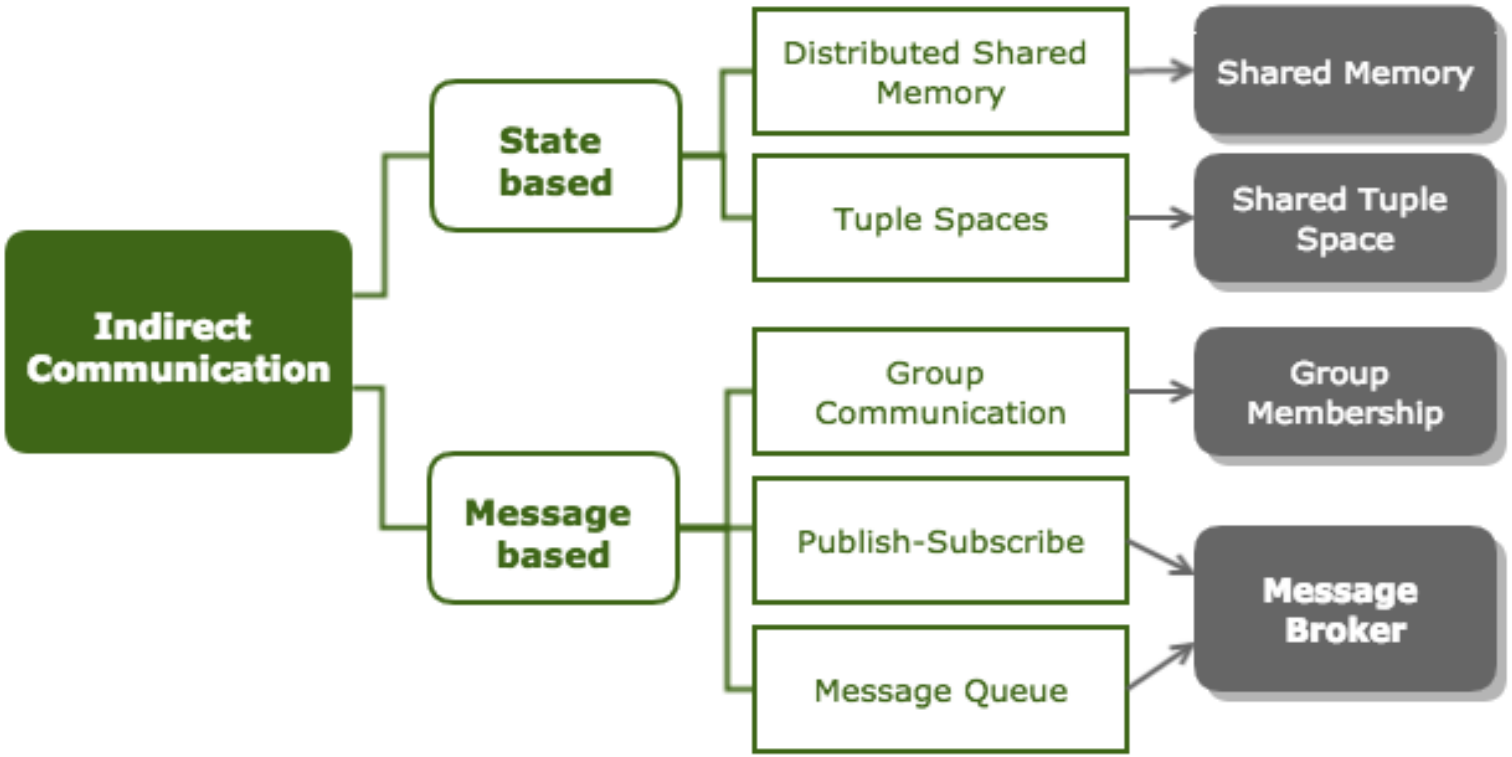
- Introduced by the **Linda** coordination language (Gelernter, Yale, 1985)
- Processes communicate by placing *tuples* in a shared tuple space and retrieving them by **content** (associative matching)
- **write** adds a tuple; **read** returns a copy of a matching tuple; **take** returns and removes it
- **read** and **take** block until a matching tuple is available
- Uncoupled in space and time; e.g. JavaSpaces, GigaSpaces

Summary of indirect communication styles

	Groups	Publish-subscribe	Message queues	DSM	Tuple spaces
Space-uncoupled	Yes	Yes	Yes	Yes	Yes
Time-uncoupled	Possible	Possible	Yes	Yes	Yes
Style of service	Communication-based	Communication-based	Communication-based	State-based	State-based
Communication pattern	1-to-many	1-to-many	1-to-1	1-to-many	1-to-1 or 1-to-many
Main intent	Reliable distributed computing	Information dissemination or EAI; mobile and ubiquitous systems	Information dissemination or EAI; commercial transaction processing	Parallel and distributed computation	Parallel and distributed computation; mobile and ubiquitous systems
Scalability	Limited	Possible	Possible	Limited	Limited
Associative	No	Content-based publish-subscribe only	No	No	Yes

Adapted from Coulouris et al., Distributed Systems, Fig. 6.27

Summary



Source: dzone.com – Message brokers in the indirect communication paradigm

Questions?

